



Name :	ID:	Section:
--------	-----	----------

Question 1 [C03] [10 Points]

Given the following **AnimalShelter** class and **Tester** class, design the **CatShelter** class so that it produces the expected output. Reuse the parent constructor and methods where necessary.

Each cat requires a strict maintenance fee of 1500 Taka per month.

Driver Code	Output
<pre> class Tester { public static void main(String[] args) { CatShelter s1 = new CatShelter("FurryGhor", 5000, 3); System.out.println(s1.getDetails()); System.out.println("-----"); s1.admitCat("Julie", 2); System.out.println("-----"); s1.admitCat("Simba", 2); System.out.println("-----"); s1.admitCat("Simba", 1); s1.admitCat("Oreo", 1); s1.updateFunds(5000); System.out.println("-----"); s1.admitCat("Oreo", 1); System.out.println("-----"); s1.admitCat("Shadow", 1); System.out.println(s1.getDetails()); } } </pre>	<pre> Shelter: FurryGhor Funds: 5000.0 Tk Occupancy: 0/3 ----- Julie admitted for 2 months. ----- Insufficient funds to keep Simba for 2 months. We can only keep this cat for 1 months. ----- Simba admitted for 1 months. Insufficient funds to keep Oreo for 1 months. ----- Oreo admitted for 1 months. ----- No space available! Shelter: FurryGhor Funds: 4000.0 Tk Occupancy: 3/3 </pre>
<pre> public class AnimalShelter { public String name; private double funds; public AnimalShelter(String name, double m) { this.name = name; this.funds = m; } public double getFunds() { return funds; } } </pre>	<pre> public void updateFunds(double amount) { this.funds += amount; } public String getDetails() { return "Shelter: " + name + "\nFunds: " + funds + " Tk"; } } </pre>



CSE111: Programming Language II (Lab)
Spring 2026
Lab Exam - 3
Duration: 20 Minutes

B

Name :	ID:	Section:
--------	-----	----------

Question 1 [C03] [10 Points]

Given the following **Cloud** class and **Tester** class, design the **Rack** class so that it produces the expected output. Reuse the parent constructor and methods where necessary.

Each VM requires a strict maintenance charge of 2000 credits per week.

Driver Code	Output
<pre>public class Tester { public static void main(String[] args) { Rack r1 = new Rack("TechNode", 6000, 3); System.out.println(r1.showDetails()); System.out.println("-----"); r1.deploy("VM1", 2); System.out.println("-----"); r1.deploy("VM2", 2); System.out.println("-----"); r1.deploy("VM2", 1); r1.deploy("VM3", 1); r1.updateCredits(5000); System.out.println("-----"); r1.deploy("VM3", 1); System.out.println("-----"); r1.deploy("VM4", 1); System.out.println(r1.showDetails()); } }</pre>	<pre>Provider: TechNode Credits: 6000.0 Servers: 0/3 ----- VM1 deployed for 2 weeks. ----- Insufficient credits to keep VM2 for 2 weeks We can only keep this server for 1 weeks ----- VM2 deployed for 1 weeks. Insufficient credits to keep VM3 for 1 weeks ----- VM3 deployed for 1 weeks. ----- No space available! Provider: TechNode Credits: 3000.0 Servers: 3/3</pre>
<pre>class Cloud { public String name; private double credits; public Cloud(String name, double c) { this.name = name; this.credits = c; } public double getCredits() { return credits; } }</pre>	<pre>public void updateCredits(double amount) { this.credits += amount; } public String showDetails() { return "Provider: " + name + "\nCredits: " + credits; } }</pre>

SET A

```
class CatShelter extends AnimalShelter {
    int capacity;
    int catCount;
    int MONTHLY_COST = 1500;

    public CatShelter(String name, double funds, int capacity) {
        super(name, funds);
        this.capacity = capacity;
        this.catCount = 0;
    }

    public void admitCat(String catName, int months) {
        double totalCost = months * MONTHLY_COST;
        double currentBalance = getFunds();

        if (catCount >= capacity) {
            System.out.println("No space available!");
        }
        else if (currentBalance >= totalCost) {
            updateFunds(-totalCost);
            catCount++;
            System.out.println(catName + " admitted for " + months + " months.");
        }
        else {
            int possibleMonths = (int) (currentBalance / MONTHLY_COST);
            System.out.println("Insufficient funds to keep " + catName + " for " + months + "
months.");
            if (possibleMonths > 0){
                System.out.println("We can only keep this cat for " + possibleMonths + " months.");
            }
        }
    }

    public String getDetails() {
        return super.getDetails() + "\nOccupancy: " + catCount + "/" + capacity;
    }
}
```

SET B

```
public class Rack extends Cloud {
    int capacity;
    int count;
    int COST = 2000;

    public Rack(String name, double c, int capacity) {
        super(name, c);
        this.capacity = capacity;
        this.count = 0;
    }

    public void deploy(String vm, int w) {
        double request = w * COST;
        double balance = getCredits();

        if (count >= capacity) {
            System.out.println("No space available!");
        }
        else if (balance >= request) {
            updateCredits(-request);
            count++;
            System.out.println(vm + " deployed for " + w + " weeks.");
        }
        else {
            int possible_weeks = (int) (balance / COST);
            System.out.println("Insufficient credits to keep " + vm + " for " + w + " weeks");
            if (possible_weeks > 0) {
                System.out.println("We can only keep this server for " + possible_weeks + "
weeks");
            }
        }
    }

    public String showDetails() {
        return super.showDetails() + "\nServers: " + count + "/" + capacity;
    }
}
```

Marking Rubric

For both sets:

Criteria	Points
Basic Class Design → [0.5] Correct class declaration with <code>extends</code> → [0.5] Proper instance variables (<code>capacity, count, COST</code>)	1
Constructor: <code>Rack / CatShelter(String name, double funds, int capacity)</code> → [0.25] Correct constructor syntax → [0.5] Correct use of <code>super()</code> to initialize parent variables → [0.5] Assign <code>capacity</code> , <code>count</code> properly	1.25
Method: <code>deploy() / admitCat(): void</code> → [0.5] Correct Parameter → [0.5] Checking the capacity → [0.5] Calculating the needed cost → [0.5] Checking the current balance → [1.5] If the balance is higher successfully admitting and printing , updating the balance and count → [1.5] If the balance is lower than the needed cost suggesting the correct months/weeks → [0.5] If the months / weeks are not zero, only then printing the suggestion → [0.5] Correct return type	6
Method: <code>showDetails() / getdetails ():String</code> → [0.25] Correct method declaration syntax → [0.5] calling the parent <code>getdetails()</code> method → [0.5] Correctly print occupancy , count → [0.25] Proper formatting → [0.25] Proper return type	1.75
Total:	10